

Rapport de Projet :

Métadonnées dans un lac de données

Encadré par :
M. STÉPHANE LOPES

Présenté par :
CHERRAB Mohamed Abdelkarim
Erkhembileg HATANBAATAR VAN

1. Introduction:

L'explosion du volume des données générées par les systèmes numériques modernes a conduit à l'émergence du Big Data, caractérisé par de grandes quantités de données variées et produites à haute vitesse. Pour répondre aux besoins de stockage et d'analyse, les architectures de type Data Lake sont devenues populaires grâce à leur flexibilité et leur capacité à stocker différents types de données.

Cependant, les Data Lakes classiques présentent plusieurs limites, notamment l'absence de gouvernance, les difficultés de gestion des données et le manque de contrôle sur les versions et les schémas. Ces problèmes rendent l'exploitation des données plus complexe et peuvent transformer les Data Lakes en espaces désorganisés appelés Data Swamps.

Dans ce contexte, les métadonnées jouent un rôle essentiel. Elles permettent de décrire, organiser et gérer les données afin d'assurer leur traçabilité, leur qualité et leur accessibilité. La gestion efficace des métadonnées est devenue indispensable dans les architectures modernes de données.

Ce projet s'intéresse à l'étude et à l'implémentation d'une architecture moderne basée sur Apache Iceberg, MinIO et un catalogue REST afin de comprendre le rôle des métadonnées dans la gouvernance et la gestion des données dans un environnement Lakehouse.

2. État de l'art :

2.1 Big Data et architectures modernes

2.2 Data Lake :

2.2.1 Définition et principes

Un lac de données est un système de stockage centralisé permettant de conserver de très grandes quantités de données dans leur format natif (structurées, semi-structurées ou non structurées). le lac de données applique le principe du schéma à la lecture (schema-on-read) : les données sont stockées brutes, et leur structure est interprétée au moment de l'analyse. Cette approche offre une grande flexibilité, autorise l'exploration de données hétérogènes et réduit les coûts d'ingestion.

2.2.2 Architecture typique :

L'architecture d'un lac de données suit souvent le principe des zones (ou couches) pour organiser le cycle de vie des données :

- Zone brute (raw / bronze) : les données sont stockées exactement comme elles sont reçues (fichiers journaux, exports bruts). Aucune transformation n'est appliquée. Cette zone sert d'archive et de point de départ pour tous les traitements ultérieurs.
- Zone de confiance (trusted / silver) : les données sont nettoyées, validées, dédoublées et éventuellement structurées (par exemple conversion en Parquet). Elles sont prêtes à être utilisées pour des analyses plus poussées.
- Zone d'analyse (curated / gold) : les données sont agrégées, enrichies et modélisées selon les besoins métier (schémas en étoile, cubes, etc.). Cette zone alimente directement les applications de reporting et de visualisation.

Les données sont généralement stockées dans des systèmes distribués comme HDFS (Hadoop Distributed File System) ou des stockages objets dans le cloud (Amazon S3, Azure Data Lake Storage, Google Cloud Storage). Les formats de fichiers les plus courants sont :

- Parquet : format colonnaire, très performant pour l'analyse.
- Avro : format de sérialisation ligne par ligne, utilisé pour les échanges.
- JSON, CSV : formats texte simples, souvent utilisés dans la zone brute.

2.2.3 Avantages des Data Lakes :

Les Data Lakes présentent plusieurs avantages qui expliquent leur adoption dans les architectures modernes de données :

- **Stockage de grands volumes de données** : ils permettent de stocker des quantités massives de données à faible coût grâce aux systèmes distribués et au stockage objet.
- **Flexibilité des formats** : un Data Lake peut contenir des données structurées, semi-structurées et non structurées, comme les fichiers CSV, JSON, images, vidéos ou logs.
- **Centralisation des données** : les données provenant de différentes sources peuvent être regroupées dans un même environnement de stockage.
- **Conservation des données brutes** : les données sont stockées dans leur format d'origine, ce qui permet de les réutiliser pour différents traitements et analyses futures.
- **Réduction des coûts** : les solutions de stockage utilisées dans les Data Lakes sont généralement moins coûteuses que les infrastructures traditionnelles de Data Warehouse.

2.2.4 Problèmes des Data Lakes :

Malgré leurs nombreux avantages, les Data Lakes classiques présentent plusieurs limitations et défis importants :

- **Manque de gouvernance** : l'absence de mécanismes efficaces de contrôle et d'organisation des données peut rendre leur gestion difficile, notamment dans les environnements contenant un grand nombre de fichiers et de sources différentes.
- **Incohérence des données** : les mises à jour, suppressions ou écritures simultanées peuvent provoquer des conflits et des incohérences dans les données stockées.
- **Absence des propriétés ACID** : les Data Lakes traditionnels ne garantissent généralement pas les propriétés ACID (Atomicity, Consistency, Isolation, Durability), ce qui limite la fiabilité des opérations transactionnelles.

- Difficulté de recherche et d'exploitation : sans gestion efficace des métadonnées, il devient difficile de localiser, comprendre et exploiter les données disponibles.
- Sécurité et contrôle d'accès limités : la gestion des permissions et de la protection des données peut devenir complexe dans les architectures distribuées.

2.3 Métadonnées :

2.3.1 Définition et rôles :

Les métadonnées peuvent être définies comme des données décrivant d'autres données. Elles fournissent des informations permettant de comprendre, localiser, organiser et exploiter efficacement les données stockées dans un système, en particulier dans un lac de données où celles-ci sont souvent conservées sous forme brute.

les métadonnées peuvent inclure :

- la structure d'un fichier,
- son emplacement de stockage,
- son origine,
- sa signification métier,
- ou encore son historique d'utilisation.

Sans métadonnées, les données deviennent difficiles à interpréter et à exploiter, ce qui peut transformer un lac de données en data swamp

2.3.2 Rôles des métadonnées :

Découverte des données : Les métadonnées facilitent la recherche et l'identification des données pertinentes au sein de volumes massifs.

Gouvernance : Elles permettent de contrôler l'utilisation des données droits d'accès et classification

Qualité des données : Les métadonnées indiquent le niveau de fiabilité ou de complétude des données. ex: date de mise à jour

2.3.3 Types de métadonnées :

Les métadonnées représentent les informations décrivant les données afin de faciliter leur gestion, leur organisation et leur exploitation. Dans les architectures modernes de données, les métadonnées jouent un rôle essentiel pour assurer la gouvernance, la traçabilité et la qualité des données. On distingue plusieurs types de métadonnées :

Métadonnées techniques :

Les métadonnées techniques décrivent les caractéristiques techniques des données. Elles incluent notamment :

- le schéma des données,
- les types de colonnes,
- le format des fichiers,
- la taille des données,
- les partitions,
- les emplacements de stockage.

Elles sont principalement utilisées par les systèmes de traitement et les moteurs analytiques.

Métadonnées métier :

Les métadonnées métier fournissent une description fonctionnelle et métier des données afin d'aider les utilisateurs à comprendre leur signification et leur utilisation. Elles peuvent contenir :

- les définitions des données,
- les règles métier,
- les descriptions des attributs,
- les indicateurs de gestion.

Ces métadonnées facilitent la collaboration entre les équipes techniques et les utilisateurs métiers.

Métadonnées opérationnelles :

Les métadonnées opérationnelles concernent les informations liées à l'utilisation et au fonctionnement des données et des systèmes. Elles incluent :

- l'historique des traitements,
- les dates de création et de modification,
- les journaux d'exécution,
- les statistiques d'utilisation,
- le suivi des accès et des performances.

Elles permettent d'assurer le suivi et le contrôle des opérations sur les données.

3. LES OUTILES :

3.1 Apache Iceberg :

Apache Iceberg est un format de table open source conçu pour les architectures modernes de type Data Lake et Lakehouse. Il a été développé afin de résoudre les limitations des Data Lakes traditionnels, notamment les problèmes de cohérence, de gestion des métadonnées et de performances dans les environnements Big Data distribués. Iceberg permet une gestion fiable et efficace des grandes tables analytiques tout en assurant la compatibilité avec plusieurs moteurs de traitement comme Apache Spark, Flink ou Trino.

3.2 Architecture d'Apache Iceberg :

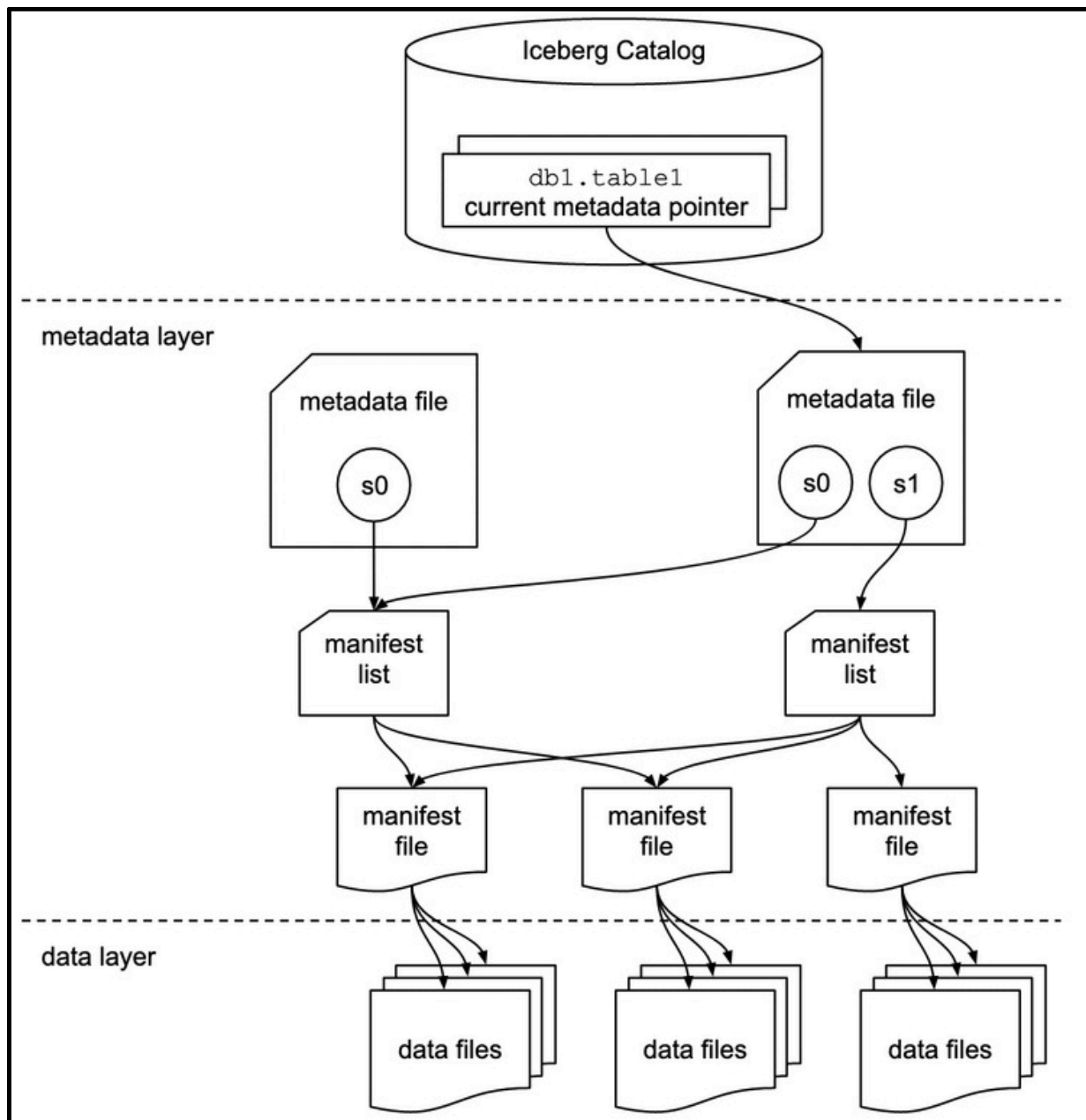
L'architecture d'Iceberg repose sur une séparation claire entre les données et les métadonnées. Les données réelles sont stockées sous forme de fichiers dans un système de stockage distribué ou objet, comme Amazon S3 ou MinIO, tandis que les métadonnées sont organisées dans plusieurs fichiers permettant de suivre l'état des tables et des versions.

Cette architecture améliore les performances des requêtes et facilite la gestion des opérations analytiques sur de grands volumes de données.

Tables Iceberg :

Une table Iceberg représente un ensemble structuré de données stockées dans des fichiers de données, généralement au format Parquet, Avro ou ORC. Chaque table possède des métadonnées décrivant son schéma, ses partitions et l'ensemble des fichiers associés.

Iceberg permet de manipuler les tables de manière transactionnelle tout en garantissant la cohérence des données.



Snapshots

Les snapshots représentent les différentes versions d'une table à un instant donné. Chaque modification effectuée sur une table, comme une insertion, une suppression ou une mise à jour, génère un nouveau snapshot.

Les snapshots permettent :

- le suivi de l'historique des données,
- la restauration d'anciennes versions,
- l'analyse des modifications effectuées sur les tables.

- **Manifest Files**

Les fichiers manifest contiennent les références vers les fichiers de données stockés dans la table. Ils enregistrent également des informations importantes comme :

- les partitions,
- les statistiques des colonnes,
- les chemins des fichiers,
- les opérations effectuées.

Les manifests permettent d'optimiser les requêtes en évitant de parcourir inutilement tous les fichiers de données.

- **Metadata Files :**

Les fichiers de métadonnées sont au cœur du fonctionnement d'Iceberg. Ils décrivent :

- le schéma de la table,
- les snapshots disponibles,
- les manifests associés,
- les configurations de partitionnement.

Le fichier principal metadata.json référence l'ensemble des informations nécessaires à la gestion de la table.

- **Évolution du schéma (Schema Evolution)**

Iceberg prend en charge l'évolution du schéma des tables sans nécessiter la réécriture complète des données. Il est possible :

- d'ajouter des colonnes,
- de renommer des attributs,
- de modifier certains types de données,
- de supprimer des colonnes.

Cette fonctionnalité facilite l'adaptation des structures de données aux besoins évolutifs des applications analytiques modernes.

• **3.2 Apache Atlas :**

Apache Atlas est une plateforme open source de gouvernance et de gestion des métadonnées, initialement développée dans le cadre de l'écosystème Hadoop/HDP. Il fournit un ensemble de capacités fondamentales pour assurer la traçabilité, la classification et la découverte des données au sein d'un Data Lake.

Définition et classification des données

Apache Atlas permet de définir des types de métadonnées personnalisés (entités, attributs, relations) grâce à un système de modélisation flexible. Les équipes de données peuvent ainsi créer des taxonomies adaptées à leur domaine métier, classer les actifs de données (tables, colonnes, fichiers, pipelines) et leur associer des tags sémantiques. Cette capacité de classification facilite notamment la conformité réglementaire, par exemple en marquant les données sensibles selon les exigences du RGPD.

Catalogue et découverte des données :

Atlas intègre un catalogue de données centralisé avec une interface de recherche permettant aux utilisateurs de retrouver facilement les actifs de données par nom, type, classification ou attribut métier. Il s'intègre nativement avec des composants Hadoop tels que Hive, HBase, Kafka, Sqoop et Falcon, automatisant ainsi la collecte des métadonnées techniques.

Architecture et intégration :

Apache Atlas repose sur une architecture orientée services qui s'appuie sur :

- **Apache Kafka** pour l'ingestion des événements de métadonnées en temps réel,
- **Apache Solr** pour l'indexation et la recherche des métadonnées,
- **JanusGraph** comme base de données de graphes pour stocker les relations entre entités.

• 3.3 Apache Parquet :

Apache Parquet est un format de stockage de données orienté colonne (columnar storage format), open source, conçu spécifiquement pour les environnements de traitement distribué comme Hadoop, Spark et les Data Lakes modernes.

Avantages dans un Data Lake :

Compression efficace : Les données similaires regroupées par colonne se compressent mieux (SNAPPY, GZIP, ZSTD)

Lecture sélective : Seules les colonnes nécessaires à une requête sont lues, réduisant les I/O

Évolution du schéma : Supporte l'ajout de nouvelles colonnes sans réécrire les fichiers existants

Supporte les types complexes : listes, maps, structures imbriquées

4 . Architecture et Solution :

4.1 Vue d'ensemble :

L'architecture du projet suit un pipeline linéaire en plusieurs étapes, allant de la génération des données brutes jusqu'à leur catalogisation dans Apache Atlas. Le flux global peut se résumer ainsi :

CSV → Parquet → Métadonnées → Apache Atlas → Classifications → Recherche

Ce pipeline a été conçu comme une preuve de concept pour démontrer le principe de gestion des métadonnées dans un Data Lake, à partir d'un jeu de données e-commerce synthétique.

4.2. Jeu de données synthétique

Le projet simule un petit lac de données e-commerce composé de six entités principales :

- Clients : données personnelles (nom, email, téléphone, adresse)
- Produits : catalogue avec prix et catégories
- Commandes : transactions entre clients et produits
- Paiements : données financières liées aux commandes
- Avis clients : champs texte libre avec notes et commentaires
- Tickets support : messages libres liés au service client

Ce jeu de données couvre plusieurs cas d'usage intéressants pour la gestion des métadonnées : données personnelles, données financières et champs texte libre, qui nécessitent chacun un traitement de classification différent.

4.3. Zone structurée — Conversion en Parquet

Les fichiers CSV sont ensuite convertis en format Apache Parquet et stockés dans la zone structurée (data/structured/). Ce passage au format Parquet apporte plusieurs avantages immédiats :

- compression efficace des données,
- lecture sélective par colonne,
- typage fort des données,
- métadonnées techniques embarquées nativement (schéma, types, statistiques).

4.4 Extraction des métadonnées techniques

Une fois les fichiers Parquet générés, le script parcourt automatiquement les fichiers et en extrait les métadonnées techniques suivantes :

- noms des colonnes et leurs types de données,
- nombre de lignes par table,
- taille des fichiers,
- schéma global de chaque table.

Ces informations sont exportées dans le fichier `metadata/metadonnees_extraites.json`, qui sert de base pour les étapes suivantes.

4.5. Génération automatique des métadonnées

À partir des fichiers Parquet générés, le système construit automatiquement deux niveaux de métadonnées : les métadonnées de tables et les métadonnées de colonnes.

Cette génération repose sur un ensemble de règles d'inférence prédéfinies. Concrètement, le système analyse les noms des tables et des colonnes, puis applique des règles basées sur des mots-clés pour en déduire automatiquement

- le domaine métier associé (CRM, Finance, Support, etc.),
- le niveau de sensibilité des données (publique, interne, confidentielle),
- les classifications pertinentes (données personnelles, RGPD, données financières, texte libre, etc.).

Par exemple, une colonne nommée email ou telephone dans la table clients déclenche automatiquement les classifications

DONNEE_PERSONNELLE et DONNEE_SENSIBLE_RGPD. De même, une colonne montant dans la table paiements est associée à la classification DONNEE_FINANCIERE.

Ces métadonnées inférées sont complétées par des fichiers YAML rédigés manuellement (metadonnees_tables.yaml et classifications_colonnes.yaml), qui apportent une couche de description métier plus fine : contexte de la table, qualité attendue des données, propriétaire, et toute information contextuelle ne pouvant pas être déduite automatiquement.

4.6. Catalogisation dans Apache Atlas

L'ensemble des métadonnées — techniques, inférées et métier — est enregistré dans Apache Atlas, qui joue le rôle de catalogue central. Cette étape se déroule en trois phases :

Création des types et classifications : avant d'enregistrer les entités, le système crée dans Atlas les types de données personnalisés et les classifications nécessaires :

- DONNEE_PERSONNELLE
- DONNEE_SENSIBLE_RGPD
- DONNEE_FINANCIERE
- EMAIL, TELEPHONE

Enregistrement des entités : les tables et leurs colonnes sont enregistrées comme entités dans Atlas, avec toutes leurs métadonnées associées.

Application des classifications : les classifications sont appliquées sur les colonnes représentatives, comme clients.email, clients.telephone, paiements.montant, avis_clients.commentaire et tickets_support.message.

La communication avec Atlas est assurée par un client Python développé dans le dossier src/, qui encapsule les appels à l'API REST d'Atlas.

- **4.7. Recherche et gouvernance dans Atlas UI :**

Une fois les métadonnées enregistrées, il est possible d'interroger le catalogue directement via l'interface web d'Apache Atlas (<http://localhost:21000>). L'utilisateur peut rechercher des actifs de données par classification, par nom de colonne ou par domaine métier, et visualiser le lignage des données.

Cette approche de recherche manuelle dans l'interface a été privilégiée pour sa stabilité et sa clarté lors des démonstrations, plutôt qu'une automatisation complète via l'API.

5. Perspective d'évolution :

5.1 Apache Iceberg :

Bien qu'Apache Iceberg ne soit pas utilisé dans la version actuelle du projet, il représente une piste d'amélioration naturelle. Son intégration permettrait d'ajouter au-dessus des fichiers Parquet une couche transactionnelle offrant la gestion des snapshots, un historique des modifications, et une évolution de schéma plus robuste — des fonctionnalités essentielles pour un Data Lake de production.

5.2 Lignage des données

Le lignage des données constitue une évolution majeure à envisager. Dans la version actuelle, les entités sont enregistrées dans Atlas sans lien explicite entre elles. L'ajout du lignage permettrait de tracer automatiquement le chemin complet d'une donnée, depuis sa source CSV brute, en passant par sa transformation en Parquet, jusqu'à son enregistrement dans Atlas. Concrètement, cela se traduirait par la création de relations de type Process dans Atlas, reliant les entités d'entrée aux entités de sortie pour chaque étape du pipeline. Cette traçabilité est particulièrement précieuse dans un contexte de conformité RGPD, car elle permet de répondre à des questions telles que : d'où vient cette donnée ? qui l'a transformée ? quels systèmes la consomment ?

5.3 API de recherche Atlas robuste

Dans la version actuelle, la recherche des métadonnées est effectuée manuellement via l'interface web d'Atlas. Une évolution importante consisterait à exploiter pleinement l'API REST d'Atlas pour automatiser et enrichir cette recherche. Cela permettrait notamment de construire des requêtes DSL ou SQL-like directement sur le catalogue, d'intégrer la recherche de métadonnées dans des pipelines de traitement automatisés, d'exposer les résultats via une interface dédiée ou un tableau de bord, et de déclencher des alertes en cas de détection de colonnes sensibles non classifiées. Cette approche rendrait le catalogue réellement exploitable à l'échelle, au-delà de la simple démonstration visuelle.

6. Conclusion :

Ce travail avait pour objectif de démontrer l'importance et la faisabilité de la gestion des métadonnées dans un Data Lake, à travers la mise en œuvre d'un pipeline complet allant de la génération des données brutes jusqu'à leur catalogisation dans Apache Atlas.

À travers ce projet, nous avons montré qu'il est possible de construire une chaîne cohérente de gestion des métadonnées en s'appuyant sur des outils open source matures. Le format Apache Parquet assure un stockage efficace et structuré des données, tandis qu'Apache Atlas joue le rôle de catalogue central permettant la description, la classification et la découverte des actifs de données. L'ajout de règles d'inférence automatiques sur les noms de tables et de colonnes démontre qu'une partie significative des métadonnées peut être produite sans intervention manuelle, réduisant ainsi la charge de travail des équipes data.

Au-delà de l'aspect technique, ce projet illustre un enjeu fondamental de la gouvernance des données : sans métadonnées bien gérées, un Data Lake devient rapidement un Data Swamp, c'est-à-dire un entrepôt de données non documentées, difficiles à retrouver, à comprendre et à exploiter en toute confiance. La mise en place d'un catalogue de métadonnées, même simplifié, constitue donc un premier pas essentiel vers une gouvernance des données maîtrisée.

En définitive, ce projet démontre qu'une approche structurée de la gestion des métadonnées, même à petite échelle, pose les fondations nécessaires à la construction d'un Data Lake fiable, gouverné et conforme aux exigences réglementaires modernes.